KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 변수, 타입, 연산, 문자열

값에 이름 붙이고, 계산하고,
보기 좋게 출력하기

```
age = 25          → 넣기
type(age)         → int
7 // 3            → 2
age >= 20         → True
f"{pi:.2f}"       → 3.14
```

오늘의 순서

1

변수에 값 넣기

= 로 넣기, 값 보기, 다시 넣기, NameError

4–9쪽

2

변수 타입

int, float, str, bool, type(), 타입 바꾸기

10–14쪽

3

사칙연산

+ - * /, 몫과 나머지, 계산 순서 네 계단과 예제, +=, 글자 연산

15–28쪽

4

비교와 논리연산

== 와 =, True와 False, and, or, not

29–37쪽

5

문자열과 출력

인덱스, 슬라이싱, sep, end, %, format(), f-string, 칸 수, 소수점, 콤마

38–52쪽

3쪽은 노트북 준비, 53–54쪽은 막힐 때와 한 장 요약이에요.

들어가기

10에서 이어서 해요

- **hello-uv 노트북, 커널 .venv** — 09 · 25–27쪽
- **변수 이름 규칙과 표기법** — 10 · 19–21쪽
- **자료형과 type()** — 10 · 22–23쪽

기억이 안 나면 그 쪽을 먼저 다시 봐요.

새 노트북 만들기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 새 파일 `basics.ipynb`
3. 오른쪽 위 커널로 .venv 고르기
4. 셀에 적고 `Shift` + `Enter` 로 실행

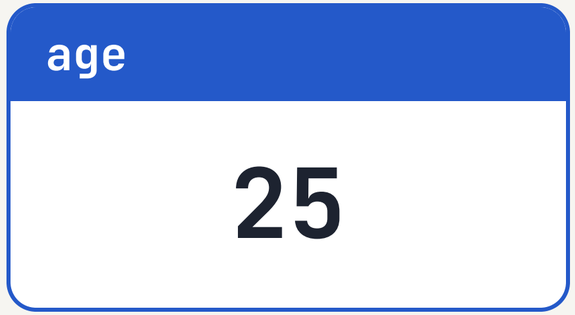
이번 챕터의 코드는 모두 노트북 셀에 적어요.

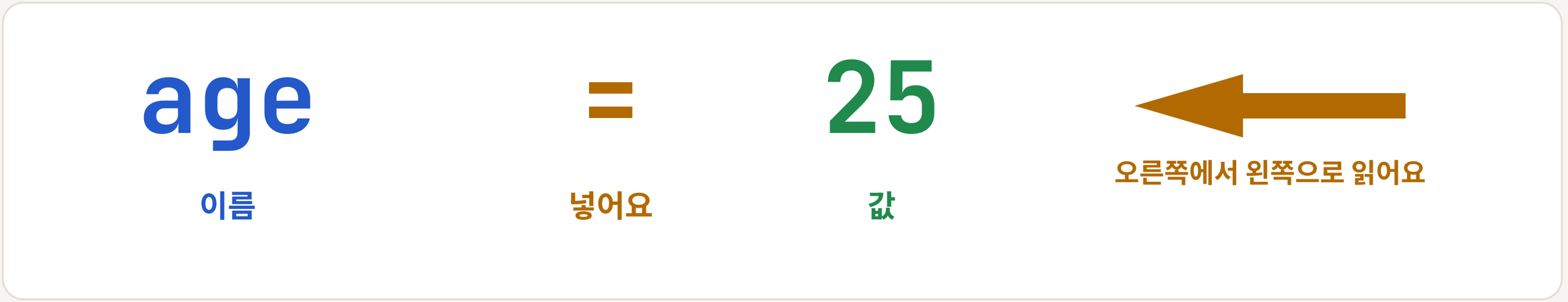
1. 변수에 값 넣기

= 는 오른쪽 값을 왼쪽 이름에 넣어요

```
[1]  age = 25                                                    Python
```

넣기만 해서 출력은 없어요. 값은 커널이 기억해요.





수학의 '같다'가 아니에요. 25를 age라는 이름에 넣어요.

1. 변수에 값 넣기

넣은 값은 두 가지 방법으로 봐요

셀 마지막 줄에 이름만 적기

[2]
```python
age
```

```
25
```

print()로 꺼내 보기

[3]
```python
print(age)
print("나이:", age)
```

```
25
나이: 25
```

이름만 적기

10에서 본 대로 셀의 마지막 줄이
값이면 print 없이도 보여 줘요.

print("설명", 값)

쉼표로 나누면 한 칸 띄워서 이어 보여 줘요.
따옴표 안 글자는 그대로, age는 값 25로 바뀌어요.

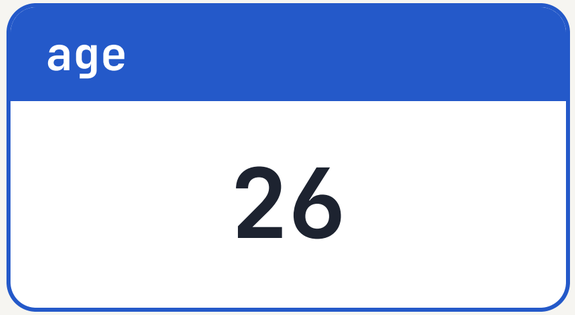
빨리 확인할 땐 이름만, 설명을 붙일 땐 print("설명", 값)을 써요.

1. 변수에 값 넣기

다시 넣으면 새 값으로 바뀌어요

[4]
```python
age = 26
age
```

```
26
```



예전 값 ~~25~~

상자 하나에는 값이 하나만 들어가요.
새 값을 넣으면 예전 값 25는 사라져요.

같은 이름에 넣을수록 마지막에 넣은 값만 남아요.

1. 변수에 값 넣기

age = age + 1은 오른쪽부터 계산해요

[5]
```
age = age + 1
age
```
Python

```
27
```

age

27

1 오른쪽 age를 읽어요

`age → 26`

2 계산해요

`26 + 1 → 27`

3 왼쪽 age에 넣어요

`age ← 27`

10에서 실행한 x = x + 1도 같은 순서로 계산했어요.

1. 변수에 값 넣기

이름만 적으면 마지막 줄만 보여요

age는 안 보여요

[6]
```python
user_name = "kim"
age
user_name
```

```
'kim'
```

print는 둘 다 보여요

[7]
```python
print(age)
print(user_name)
```

```
27
kim
```

가운데 줄 age는 실행은 되지만 화면에 나오지 않아요.

`'kim'`	이름만 적기는 따옴표까지
`kim`	print는 글자만

여러 값을 보고 싶으면 줄마다 print()를 써요.

1. 변수에 값 넣기

이름을 틀리게 적으면 NameError가 나요

```
[8]  Age                                                    Python
```

```
NameError: name 'Age' is not defined
```

대문자와 소문자는 다른 이름이에요

`age` 넣은 이름

`Age` 한 번도 넣은 적 없는 이름

이럴 때도 같은 오류가 나요

- 철자를 틀렸어요 (agee)
- 넣는 셀을 아직 실행하지 않았어요
- Restart 뒤에 아래 셀만 실행했어요

NameError가 나면 철자부터 보고, 넣는 셀을 실행했는지 봐요.

2. 변수 타입

값마다 타입이 있어요

[1]
```python
age = 25
height = 172.5
user_name = "kim"
is_member = False
print(type(age))
print(type(height))
print(type(user_name))
print(type(is_member))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

정수 int

`25`

소수점 없는 숫자

실수 float

`172.5`

소수점 있는 숫자

문자열 str

`"kim"`

따옴표로 감싼 글자

불리언 bool

`False`

True 또는 False

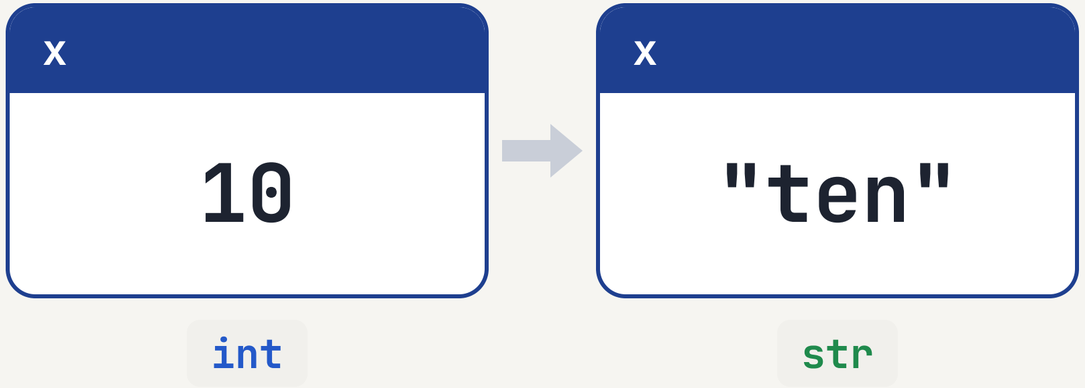
타입은 값의 종류예요. 10에서 본 자료형과 같은 말이에요.

2. 변수 타입

타입은 이름이 아니라 값을 따라가요

[2]
```python
x = 10
print(type(x))
x = "ten"
print(type(x))
```

```
<class 'int'>
<class 'str'>
```



이름 x는 그대로인데, 넣은 값이 바뀌니 타입도 바뀌었어요.
타입을 미리 정하지 않아도 넣는 값을 보고 정해져요.

한 이름에는 한 가지 뜻의 값만 넣어요. 헷갈리면 type()으로 확인해요.

2. 변수 타입

따옴표 하나로 숫자가 글자가 돼요

[3]
```python
print(type(25))
print(type("25"))
print(25 + 1)
print("25" + "1")
```

```
<class 'int'>
<class 'str'>
26
251
```

25 int

25 + 1 → **26**

숫자끼리 더해요

"25" str

"25" + "1" → **251**

글자끼리 이어 붙여요

겉모습이 숫자여도 따옴표 안에 있으면 str이에요.

2. 변수 타입

int(), float(), str()로 타입을 바꿔요

[4]
```python
print(int("3") + 3)
print(str(3) + "3")
print(float("3.5"))
print(int(3.9))
```

```
6
33
3.5
3
```

글자를 정수로

`int("3")` → `3`

숫자를 글자로

`str(3)` → `'3'`

글자를 실수로

`float("3.5")` → `3.5`

소수점 아래는 버려요

`int(3.9)` → `3`

바꾸고 싶은 타입 이름을 함수처럼 쓰고 괄호 안에 값을 넣어요.

2. 변수 타입

숫자로 못 읽는 글자는 ValueError가 나요

```
[5] int("삼")                                                    Python
```

```
ValueError: invalid literal for int() with base 10: '삼'
```

```
[6] int("3.5")                                                   Python
```

```
ValueError: invalid literal for int() with base 10: '3.5'
```

한글 숫자는 못 읽어요

int()는 0부터 9까지 숫자 글자만 읽어요.
"삼"은 사람에게만 숫자예요.

소수점 글자는 float()로

`float("3.5")` → **3.5**

앞 장에서 실행한 결과예요

ValueError는 타입은 맞는데 값을 바꿀 수 없다는 뜻이에요.

3. 사칙연산

더하기, 빼기, 곱하기, 나누기

[1]
```python
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 3)
```

```
10
4
21
2.3333333333333335
```

+ 더하기

`7 + 3` → `10`

- 빼기

`7 - 3` → `4`

***** 곱하기

× 대신 별표

`7 * 3` → `21`

/ 나누기

÷ 대신 빗금

`7 / 3` → `2.333…`

키보드에 ×, ÷가 없어서 곱하기는 *, 나누기는 / 를 써요.

3. 사칙연산

/ 로 나누면 항상 소수점이 붙어요

```
[2]  6 / 2                                    Python

     3.0
```

딱 나눠떨어져도

`6 / 2` → `3.0`

3이 아니라 3.0이에요.

3 **정수 int**
소수점 없는 숫자

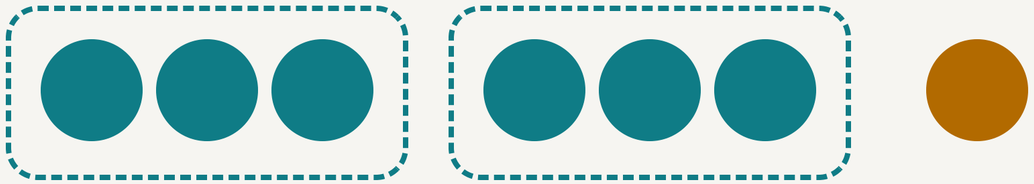
3.0 **실수 float**
소수점 있는 숫자. / 의 결과는 늘 이쪽

값은 같아도 타입이 달라요. 2파트에서 본 int와 float예요.

3. 사칙연산

몫 //, 나머지 %, 거듭제곱 **

사탕 7개를 3개씩 나누면



몫 2 (묶음 수)　　**나머지 1**

[3]
```
print(7 // 3)
print(7 % 3)
print(2 ** 3)
```
Python

```
2
1
8
```

//　몫

7 // 3　→　2

나누고 소수점 아래는 버려요

%　나머지

7 % 3　→　1

나누고 남은 수

****　거듭제곱**

2 ** 3　→　8

2 × 2 × 2

% 는 퍼센트가 아니라 나머지예요.

3. 사칙연산

곱하기를 더하기보다 먼저 계산해요

[4]
```python
2 + 3 * 4
```
Python

```
14
```

[5]
```python
(2 + 3) * 4
```
Python

```
20
```

`2 + 3 * 4` 3 * 4 먼저

`2 + 12` 그다음 더하기

`14`

`(2 + 3) * 4` 괄호 먼저

`5 * 4` 그다음 곱하기

`20`

먼저 하는 순서 `( )` → `**` → `* / // %` → `+ -`

헷갈리면 먼저 할 계산을 괄호로 묶어요.

3. 사칙연산

계산 순위는 네 단계예요

1 ( ) **괄호**
가장 먼저

2 ** **거듭제곱**

3 * / // % **곱하기, 나누기, 몫, 나머지**
넷은 같은 순위

4 + - **더하기, 빼기**
가장 나중

몫과 나머지는

// 와 % 는 곱하기, 나누기와 같은 3단계예요.

** 는 그보다 한 단계 위라서 먼저 계산해요.

같은 단계끼리는 왼쪽부터 풀어요.

위 계단부터 풀어요. 같은 계단에서는 왼쪽부터예요.

3. 사칙연산

** 는 곱하기와 몫보다 먼저 계산해요

[6]
```python
print(2 ** 3 // 3)
print(2 * 3 ** 2)
print(-2 ** 2)
print((-2) ** 2)
```

```
2
18
-4
4
```

`2 ** 3 // 3` ** 먼저

`8 // 3` 그다음 몫

`2`

`2 * 3 ** 2` ** 먼저

`2 * 9` 그다음 곱하기

`18`

`-2 ** 2` 2 ** 2 먼저

`-4` 빼기 기호는 나중

`(-2) ** 2` 괄호 먼저

`4`

-2 ** 2는 -4예요. 음수를 거듭제곱하려면 (-2)처럼 괄호로 묶어요.

3. 사칙연산

같은 순위끼리는 왼쪽부터 풀어요

[7]
```python
print(10 - 4 + 2)
print(8 / 2 * 4)
print(7 // 2 * 2)
```

```
8
16.0
6
```

`10 - 4 + 2`

왼쪽 빼기 먼저

`6 + 2`

`8`

⚠ 10 - 6 = 4가 아니에요

`8 / 2 * 4`

왼쪽 나누기 먼저

`4.0 * 4`

`16.0`

⚠ 8 / 8 = 1이 아니에요

`7 // 2 * 2`

왼쪽 몫 먼저

`3 * 2`

`6`

⚠ // 와 * 도 같은 순위

8 ÷ 2 × 4를 수학 시간처럼 앞에서부터 차례로 계산해요. 파이썬도 똑같아요.

뒤쪽을 먼저 하고 싶으면 그 부분을 괄호로 묶어요.

3. 사칙연산

섞여 있으면 한 계단씩 풀어요

[8]
```python
print(10 - 2 * 3 ** 2 // 4)
```

```
6
```

`10 - 2 * 3 ** 2 // 4` 시작

`10 - 2 * 9 // 4` 2단계 ** · 3 ** 2 = 9

`10 - 18 // 4` 3단계 왼쪽부터 · 2 * 9 = 18

`10 - 4` 3단계 · 18 // 4 = 4

`6` 4단계 · 10 - 4 = 6

직접 짤 땐 10 - (2 * 3 ** 2) // 4처럼 괄호로 순서를 보여 줘요.

계단 순서

1. ( )
2. **
3. * / // %
4. + -

괄호가 없으니 1단계는 건너뛰어요.

3. 사칙연산

괄호 하나로 답이 달라져요 1 · 더하기와 곱하기

[9]
```python
print(10 + 2 * 3)
print((10 + 2) * 3)
print(20 - 10 / 2)
print((20 - 10) / 2)
print(12 / 3 * 2)
print(12 / (3 * 2))
print(100 - 30 - 20)
print(100 - (30 - 20))
```

```
16
36
15.0
5.0
8.0
2.0
50
90
```

괄호 없이	괄호를 치면
`10 + 2 * 3` → `16`	`(10 + 2) * 3` → `36`
곱하기 먼저 6, 더해서 16 · 괄호면 12를 먼저, 곱해서 36	
`20 - 10 / 2` → `15.0`	`(20 - 10) / 2` → `5.0`
나누기 먼저 5.0, 빼서 15.0 · 괄호면 10을 먼저, 나눠서 5.0	
`12 / 3 * 2` → `8.0`	`12 / (3 * 2)` → `2.0`
같은 순위라 왼쪽부터 4.0 * 2 · 괄호면 12 / 6	
`100 - 30 - 20` → `50`	`100 - (30 - 20)` → `90`
왼쪽부터 70 - 20 · 괄호면 100 - 10	

같은 숫자라도 괄호 자리에 따라 결과가 달라져요.

3. 사칙연산

괄호 하나로 답이 달라져요 2 · 거듭제곱, 몫, 나머지

[10]
```
print(2 ** 2 * 3)
print(2 ** (2 * 3))
print(17 % 5 * 2)
print(17 % (5 * 2))
print(3 + 10 // 3)
print((3 + 10) // 3)
print(-3 ** 2)
print((-3) ** 2)
```
Python

```
12
64
4
7
6
4
-9
9
```

괄호 없이	괄호를 치면
`2 ** 2 * 3` → 12	`2 ** (2 * 3)` → 64
** 먼저 4, 곱해서 12 · 괄호면 2 ** 6	
`17 % 5 * 2` → 4	`17 % (5 * 2)` → 7
같은 순위라 왼쪽부터 2 * 2 · 괄호면 17 % 10	
`3 + 10 // 3` → 6	`(3 + 10) // 3` → 4
몫 먼저 3, 더해서 6 · 괄호면 13 // 3	
`-3 ** 2` → -9	`(-3) ** 2` → 9
** 가 빼기 기호보다 먼저 · 괄호면 -3을 제곱	

%, // 는 곱하기와 같은 순위, ** 는 그보다 먼저예요.

3. 사칙연산

직접 풀어 봐요

1 `7 // 2 ** 2`

```
1. 2 ** 2 = 4
2. 7 // 4 = 1
```

1

2 `10 % 3 ** 2`

```
1. 3 ** 2 = 9
2. 10 % 9 = 1
```

1

3 `2 + 3 * 4 ** 2 // 8`

```
1. 4 ** 2 = 16
2. 3 * 16 = 48
3. 48 // 8 = 6
4. 2 + 6 = 8
```

8

4 `50 - 5 * 2 ** 3 % 7`

```
1. 2 ** 3 = 8
2. 5 * 8 = 40
3. 40 % 7 = 5
4. 50 - 5 = 45
```

45

셀에 print()로 적고 실행해서 답을 맞춰 봐요.

3. 사칙연산

숫자에 이름을 붙여 계산해요

장보기 합계

[12]
```python
price = 3000
count = 2
total = price * count
print("합계:", total)
```

```
합계: 6000
```

세 사람이 나눠 내기

[13]
```python
total = 10000
people = 3
print("한 사람:", total // people)
print("남는 돈:", total % people)
```

```
한 사람: 3333
남는 돈: 1
```

`3000 * 2` 무슨 숫자인지 몰라요

`price * count` 가격 곱하기 개수

price만 바꾸고 다시 실행하면 합계도 새로 계산돼요.

3. 사칙연산

+= 는 지금 값에 더해서 다시 넣어요

[14]
```python
score = 0
score += 10
score += 5
score
```

15



`score += 10` 는 `score = score + 10` 를 줄인 거예요

같은 모양 `-=` `*=` `/=` 도 있어요

점수나 합계처럼 값을 쌓아 갈 때 써요.

3. 사칙연산

글자에도 + 와 * 를 써요

이어 붙여요

```
"kim" + "lee"
```

→ `'kimlee'`

3번 반복해요

```
"ha" * 3
```

→ `'hahaha'`

따옴표가 있으면 숫자도 글자예요

```
"3" + "3"
```

→ `'33'`

글자와 숫자는 더할 수 없어요

```
"3" + 3
```

→ `TypeError: can only concatenate str (not "int") to str`

글자와 숫자를 섞어 더하면 TypeError예요. int("3") + 3처럼 타입을 맞춰요.

4. 비교와 논리연산

비교하면 답은 True 아니면 False예요

[1]
```python
age = 25
print(age > 20)
print(age < 20)
print(age == 25)
print(age != 25)
print(age >= 25)
print(age <= 24)
```

```
True
False
True
False
True
False
```

>	크다	age > 20	True
<	작다	age < 20	False
==	같다	age == 25	True
!=	다르다	age != 25	False
>=	크거나 같다	age >= 25	True
<=	작거나 같다	age <= 24	False

결과는 10에서 본 불리언(bool)이에요. 같다는 = 를 두 번 써요.

4. 비교와 논리연산

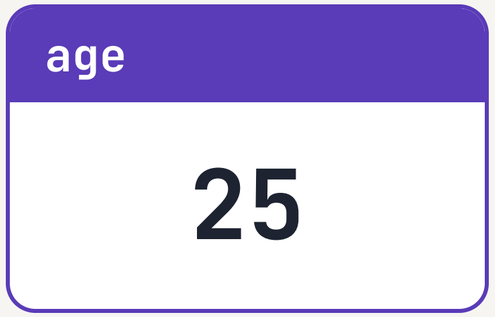
= 하나는 넣기, == 두 개는 묻기

== 묻기

[2]
```
age == 30
```
Python

```
False
```

age가 30이냐고 물어요.
age는 25 그대로예요.

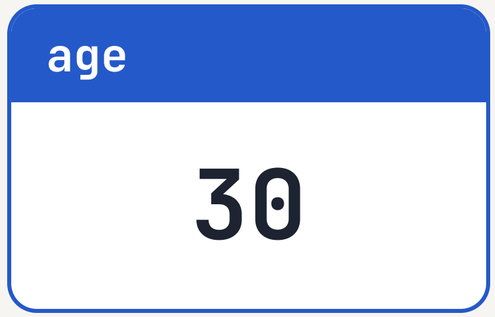


= 넣기

[3]
```
age = 30
age
```
Python

```
30
```

age에 30을 넣어요. 값이 바뀌어요.



비교하려고 = 를 하나만 쓰면 값이 바뀌어 버려요.

4. 비교와 논리연산

비교한 결과도 변수에 넣을 수 있어요

[4]
```python
age = 25
is_adult = age >= 20
print(is_adult)
print(type(is_adult))
```

```
True
<class 'bool'>
```

이름은 is_ 로 시작해요

10의 is_alive처럼 True, False를 담는
변수는 is_adult로 지으면 읽기 쉬워요.

`age >= 20`

25 >= 20 을 먼저 계산해요

↓ `True`

is_adult

True

= 오른쪽을 먼저 계산한다는 규칙은 비교에서도 같아요.

and 는 둘 다 True일 때만 True예요

[5]
```
print(True and True)
print(True and False)
print(False and True)
print(False and False)
```
Python

```
True
False
False
False
```

왼쪽		오른쪽		결과
True	and	True	→	True
True	and	False	→	False
False	and	True	→	False
False	and	False	→	False

하나라도 False면 결과는 False예요.

4. 비교와 논리연산

or 는 하나만 True여도 True예요

[6]
```python
print(True or True)
print(True or False)
print(False or True)
print(False or False)
```

```
True
True
True
False
```

왼쪽		오른쪽	결과
True	or	True	→ True
True	or	False	→ True
False	or	True	→ True
False	or	False	→ False

둘 다 False일 때만 False예요.

4. 비교와 논리연산

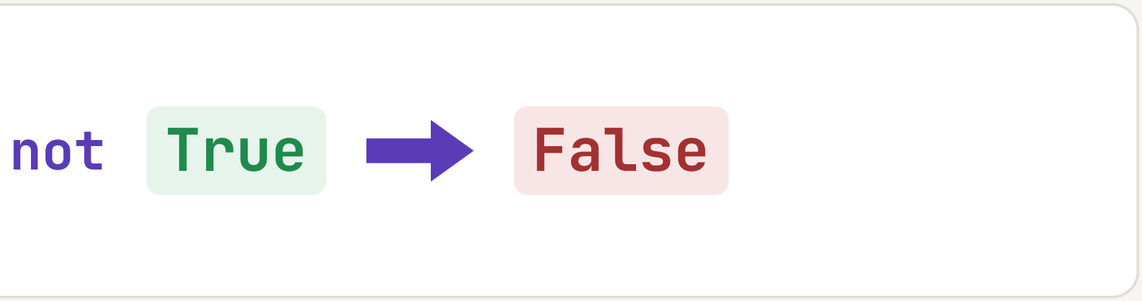
not 은 True와 False를 뒤집어요

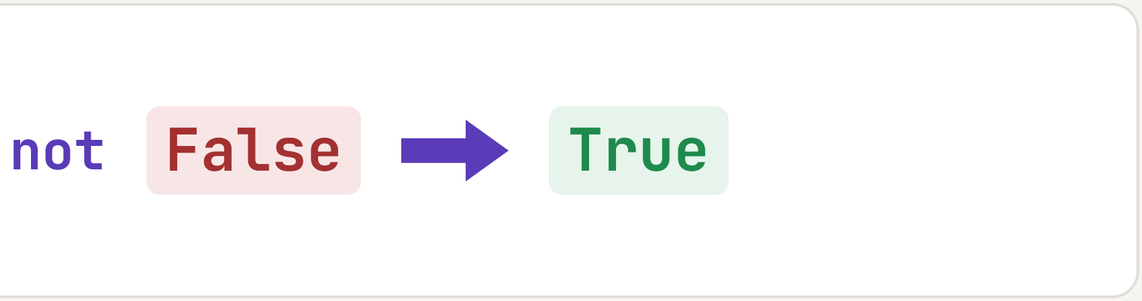
[7]
```python
print(not True)
print(not False)
```

```
False
True
```

앞에 한 번만 붙여요

and, or는 값 두 개 사이에 두고,
not은 값 하나 앞에 붙여요.





not 은 '아니다'예요. 바로 뒤의 값을 반대로 바꿔요.

4. 비교와 논리연산

놀이기구 탑승 조건을 and로 적어요

조건 · 키 120cm 이상 **그리고** 7살 이상

[8]
```python
height = 130
age = 9
height >= 120 and age >= 7
```

```
True
```

130 >= 120 → True

9 >= 7 → True

True and True → True

탈 수 있어요

[9]
```python
height = 110
height >= 120 and age >= 7
```

```
False
```

110 >= 120 → False

9 >= 7 → True

False and True → False

키가 모자라서 못 타요

조건을 모두 만족해야 하면 and예요.

4. 비교와 논리연산

할인 조건을 or로 적어요

조건 · 회원이거나 **또는** 5만 원 이상 샀으면 할인

[10]
```python
is_member = False
total = 60000
is_member or total >= 50000
```

```
True
```

`is_member` → `False`

`60000 >= 50000` → `True`

`False or True` → `True`

회원이 아니어도 할인돼요

[11]
```python
not is_member
```

```
True
```

`not is_member`

`not False` → `True`

회원이 아닌가요?

not을 붙이면 뜻도 반대로 읽어요.

둘 중 하나만 만족해도 되면 or예요.

4. 비교와 논리연산

계산, 비교, not, and, or 순서로 풀어요

[14]
```python
2 + 3 > 4 and not False
```
Python

```
True
```

시작 `2 + 3 > 4 and not False`

1 계산 `5 > 4 and not False`

2 비교 `True and not False`

3 not `True and True`

4 and `True`

먼저 하는 순서

1. `+ - * /` 계산
2. `> == !=` 비교
3. `not` 뒤집기
4. `and` 그리고
5. `or` 또는

헷갈리면 (age >= 20) and (age < 30)처럼 괄호로 묶어요.

5. 문자열과 출력

문자열은 따옴표로 감싸요

[1]
```python
a = 'kim'
b = "kim"
print(a == b)
print(len("hello"))
print("It's me")
```

```
True
5
It's me
```

`'kim'  "kim"`

작은따옴표, 큰따옴표 모두 같은 문자열

a == b 는 True

`len("hello")`

글자 수를 세요

h e l l o 다섯 글자라서 5

`"It's me"`

글자 안에 ' 가 있으면

바깥을 " 로 감싸요

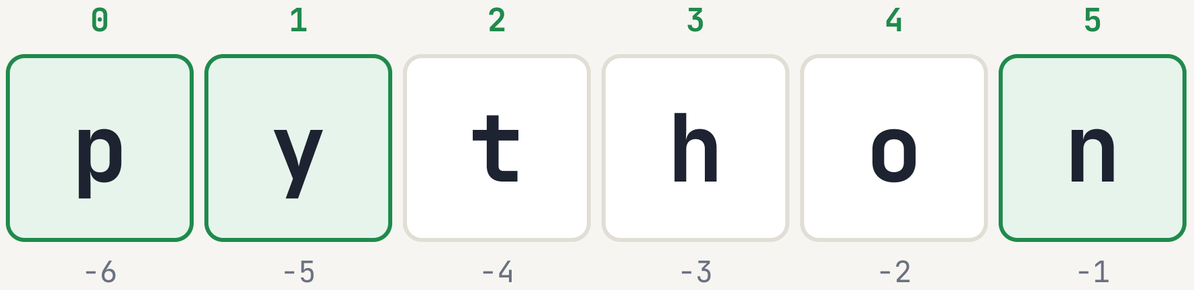
한 파일 안에서는 한 가지 따옴표로 맞춰 쓰면 읽기 쉬워요.

5. 문자열과 출력

[번호]로 글자 하나를 꺼내요

[2]
```python
word = "python"
print(word[0])
print(word[1])
print(word[-1])
```

```
p
y
n
```



위 숫자는 앞에서부터, 아래 숫자는 뒤에서부터 센 번호예요.

[5]
```python
word[6]
```

```
IndexError: string index out of range
```

0 번호는 1이 아니라 0부터 세요. 여섯 글자의 마지막 번호는 5예요.

word[-1]은 맨 뒤 글자예요. 길이를 몰라도 끝을 꺼낼 수 있어요.

5. 문자열과 출력

[시작:끝]으로 여러 글자를 잘라요

[3]
```
word[0:2]
```
Python

```
'py'
```

[4]
```
print(word[2:5])
print(word[:2])
print(word[2:])
```
Python

```
tho
py
thon
```

word[0:2]
0번부터 2번 앞까지



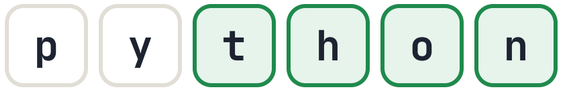
word[2:5]
2번부터 5번 앞까지



word[:2]
시작을 비우면 처음부터



word[2:]
끝을 비우면 끝까지



끝 번호의 글자는 들어가지 않아요. [0:2]는 0번, 1번 두 글자예요.

5. 문자열과 출력

print의 sep과 end로 모양을 바꿔요

[6]
```python
print("kim", "lee")
print("kim", "lee", sep="-")
print("2026", "11", "02", sep=".")
print("hello", end=" ")
print("world")
```

Python

```
kim lee
kim-lee
2026.11.02
hello world
```

sep 값 사이에 넣을 글자

`sep="-"  →  kim-lee`

기본은 빈칸 한 개

end 줄 끝에 붙일 글자

`end=" "  →  다음 print가 같은 줄에`

기본은 줄바꿈

print("설명", 값)이 빈칸으로 이어진 건 sep의 기본값이 빈칸이라서예요.

5. 문자열과 출력

\ 로 시작하는 특수 문자

[7]
```python
print("첫째 줄\n둘째 줄")
print("이름\tkim")
print("경로: C:\\Users")
```

```
첫째 줄
둘째 줄
이름	kim
경로: C:\Users
```

\n 줄바꿈
글자 사이에서 다음 줄로

\t 탭
칸을 띄워 줄 맞추기

**** 역슬래시 하나
\ 자체를 쓰고 싶을 때

Windows 키보드에서는 \ 가 ₩ 로 보여요. 같은 키예요.

5. 문자열과 출력

값을 끼워 넣는 방법은 세 가지예요

[8]
```python
name = "kim"
age = 25
print("%s 님은 %d살이에요" % (name, age))
print("{} 님은 {}살이에요".format(name, age))
print(f"{name} 님은 {age}살이에요")
```

```
kim 님은 25살이에요
kim 님은 25살이에요
kim 님은 25살이에요
```

% 포맷팅

`"%s" % name`

C 언어에서 온 가장 옛 방식

옛 코드와 책에서 자주 봐요

format()

`"{}".format(name)`

파이썬 3.0부터 (2008년)

중간 세대예요

f-string

`f"{name}"`

파이썬 3.6부터 (2016년)

요즘 표준. 새 코드는 이걸로 써요

세 줄 모두 결과가 같아요. 이 챕터는 % 와 f-string을 나란히 봐요.

5. 문자열과 출력

%s, %d 자리와 f-string의 { }

% 방식 · 옛 방식

[9]
```python
print("이름: %s" % name)
print("나이: %d살" % age)
```

```
이름: kim
나이: 25살
```

f-string · 요즘 방식

[10]
```python
print(f"이름: {name}")
print(f"나이: {age}살")
print(f"내년: {age + 1}살")
```

```
이름: kim
나이: 25살
내년: 26살
```

`"%s" % name`	→ `f"{name}"`	문자열 자리
`"%d" % age`	→ `f"{age}"`	정수 자리. f-string은 타입을 안 적어도 돼요
`"%s %d" % (name, age)`	→ `f"{name} {age}"`	여러 개면 % 는 괄호로 묶어 순서대로

f-string은 따옴표 앞에 f, 중괄호 안에 이름이나 계산식을 바로 적어요.

5. 문자열과 출력

format()은 {0} {1} {2} 번호로 값을 넣어요

[11]
```python
cake = "치즈케이크"
price = 5000
qty = 2
print("주문내역은 : {0}, {1}, {2}".format(cake, price, qty))
print("주문내역은 : {}, {}, {}".format(cake, price, qty))
print("{0} {2}개, 하나에 {1}원".format(cake, price, qty))
print("{0}, {0}, {0}".format(cake))
```

Python

```
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
치즈케이크 2개, 하나에 5000원
치즈케이크, 치즈케이크, 치즈케이크
```

{0} ← **cake**

0번째 값 → 치즈케이크

{1} ← **price**

1번째 값 → 5000

{2} ← **qty**

2번째 값 → 2

번호는 0부터 세요. 번호를 비우면 순서대로, 번호를 쓰면 순서를 바꾸거나 여러 번 쓸 수 있어요.

5. 문자열과 출력

같은 주문내역을 세 방식으로 적어 봐요

[12]
```python
print("주문내역은 : %s, %d, %d" % (cake, price, qty))
print("주문내역은 : {0}, {1}, {2}".format(cake, price, qty))
print(f"주문내역은 : {cake}, {price}, {qty}")
print(f"{cake} {qty}개, 하나에 {price}원")
print(f"{cake} {qty}개, 합계 {price * qty:,}원")
```

```
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
주문내역은 : 치즈케이크, 5000, 2
치즈케이크 2개, 하나에 5000원
치즈케이크 2개, 합계 10,000원
```

% 방식

`%s, %d, %d`

자리 순서 = 값 순서

format()

`{0}, {1}, {2}`

번호로 값을 골라요

f-string

`{cake}, {price}, {qty}`

이름을 바로 적어요

f-string은 { } 안에 계산식과 모양도 넣어요. {price * qty:,} 는 합계에 콤마를 찍어요.

5. 문자열과 출력

%5d 와 {42:5d} 는 다섯 칸을 확보해요

% 방식 · 옛 방식

[13]
```
print("[%5d]" % 42)
print("[%-5d]" % 42)
print("[%05d]" % 42)
print("[%5s]" % "kim")
```
Python

```
[   42]
[42   ]
[00042]
[  kim]
```

f-string · 요즘 방식

[14]
```
print(f"[{42:5d}]")
print(f"[{42:<5d}]")
print(f"[{42:05d}]")
print(f"[{'kim':>5}]")
print(f"[{'kim':5}]")
```
Python

```
[   42]
[42   ]
[00042]
[  kim]
[kim  ]
```

`%5d`	`{42:5d}`	□ □ □ 4 2	다섯 칸, 오른쪽 맞춤
`%-5d`	`{42:<5d}`	4 2 □ □ □	왼쪽 맞춤. % 는 -, f-string은 <
`%05d`	`{42:05d}`	0 0 0 4 2	빈칸을 0으로
`%5s`	`{'kim':>5}`	□ □ k i m	글자 오른쪽 맞춤. f-string은 > 를 붙여요

f-string에서 글자는 기본이 왼쪽 맞춤이에요. {'kim':5}는 [kim]이 돼요.

5. 문자열과 출력

%0.2f 와 {pi:.2f} 는 소수점 두 자리예요

% 방식 · 옛 방식

[15]
```python
pi = 3.141592
print("%f" % pi)
print("%.2f" % pi)
print("%0.2f" % pi)
print("%.4f" % pi)
print("[%8.2f]" % pi)
```

```
3.141592
3.14
3.14
3.1416
[    3.14]
```

f-string · 요즘 방식

[16]
```python
print(f"{pi}")
print(f"{pi:.2f}")
print(f"{pi:0.2f}")
print(f"{pi:.4f}")
print(f"[{pi:8.2f}]")
```

```
3.141592
3.14
3.14
3.1416
[    3.14]
```

`%0.2f` `{pi:0.2f}`

0 전체 칸 수 **.2** 소수점 아래 자리 **f** 실수

0은 최소 칸 수라 영향이 없어서 0.2f 와 .2f 는 결과가 같아요.

5. 문자열과 출력

가격은 {price:,} 로 천 단위 콤마를 찍어요

[17]
```python
price = 1234567
print(f"{price:,}")
print(f"{price:,}원")
print(f"{1234567.891:,.2f}")
fee = 35000
print(f"[{fee:8,}]")
```

Python

```
1,234,567
1,234,567원
1,234,567.89
[  35,000]
```

`{price:,}` → 1,234,567

콤마만

`{x:,.2f}` → 1,234,567.89

콤마와 소수점 두 자리

`{fee:8,}` → 35,000

여덟 칸 확보 + 콤마

[18]
```python
print("%,d" % price)
```

Python

```
ValueError: unsupported format character ',' (0x2c) at index 1
```

% 방식에는 콤마 옵션이 없어요. 콤마가 필요하면 f-string을 써요.

콤마는 보여 주는 모양만 바꿔요. price 값은 그대로 1234567이에요.

5. 문자열과 출력

format()도 콜론 뒤 모양은 f-string과 같아요

[19]

```python
price = 5000
print("[{0:5d}]".format(qty))
print("{0:.2f}".format(3.141592))
print("{0:,}원".format(1234567))
print("{0}: {1:,}원 x {2}개".format(cake, price, qty))
```

```
[    2]
3.14
1,234,567원
치즈케이크: 5,000원 x 2개
```

`{0:5d}`	→	`{qty:5d}`	다섯 칸 확보
`{0:.2f}`	→	`{pi:.2f}`	소수점 두 자리
`{1:,}`	→	`{price:,}`	천 단위 콤마

f-string은 이름을, format()은 번호를 콜론 앞에 적어요. 콜론 뒤는 똑같아요.

5. 문자열과 출력

칸을 맞추면 표처럼 보여요

% 방식 · 옛 방식

[20]
```python
print("%-6s %5d" % ("kim", 90))
print("%-6s %5d" % ("lee", 100))
print("%-6s %5d" % ("park", 7))
print("할인 %d%%" % 30)
```

```
kim        90
lee       100
park        7
할인 30%
```

f-string · 요즘 방식

[21]
```python
print(f"{'kim':<6} {90:5d}")
print(f"{'lee':<6} {100:5d}")
print(f"{'park':<6} {7:5d}")
print(f"할인 {30}%")
```

```
kim        90
lee       100
park        7
할인 30%
```

`%-6s`	→ `{:<6}`	여섯 칸 왼쪽 맞춤 (이름)
`%5d`	→ `{:5d}`	다섯 칸 오른쪽 맞춤 (점수)
`%%`	→ `%`	% 글자 그대로. % 방식만 두 번 적어요

성적표, 영수증처럼 줄을 맞춰 보여 줄 때 칸 수를 써요.

5. 문자열과 출력

자리와 값이 안 맞으면 오류가 나요

[22]
```python
"%d살" % "kim"
```
```
TypeError: %d format: a real number is required, not str
```

% 방식
%d 자리에 글자를 넣었어요

[23]
```python
"%s 님은 %d살" % name
```
```
TypeError: not enough arguments for format string
```

% 방식
자리는 둘, 값은 하나예요

[24]
```python
"%s" % (name, age)
```
```
TypeError: not all arguments converted during string formatting
```

% 방식
자리는 하나, 값은 둘이에요

[25]
```python
f"{name:d}"
```
```
ValueError: Unknown format code 'd' for object of type 'str'
```

f-string
글자에 정수 모양 d를 줬어요

% 는 자리 수와 값 수를, f-string은 콜론 뒤 모양과 타입을 먼저 봐요.

정리

막히면 오류의 마지막 줄을 봐요

1	`NameError: name 'Age' is not defined` 대문자와 소문자를 다르게 적었거나 넣는 셀을 안 돌렸어요	철자 확인, Run All
2	`NameError: name 'true' is not defined` True, False는 첫 글자가 대문자예요	True False
3	`TypeError: can only concatenate str (not "int") to str` 따옴표 글자와 숫자를 더했어요	int("3") + 3
4	`ValueError: invalid literal for int() with base 10: '삼'` int()가 숫자로 못 읽는 글자예요	소수 글자는 float()
5	`ZeroDivisionError: division by zero` 0으로 나눴어요	나누는 수 확인
6	`TypeError: not enough arguments for format string` % 자리 수와 값 수가 달라요	f-string으로
7	**비교했는데 값이 바뀌었어요** = 를 하나만 썼어요	== 두 개

오류가 나면 위쪽 Traceback보다 맨 아래 줄부터 읽어요.

한 장 요약

`age = 25`

오른쪽 값을 왼쪽 이름에 넣기

`f"{name} {age}살"`

값을 끼워 넣기

`type(age)`

타입 확인

`int("3")  str(3)`

타입 바꾸기

`+ - * /`

사칙연산. / 는 항상 실수

`// % **`

몫, 나머지, 거듭제곱

`f"{price:,}원"`

콤마 찍어 출력

`== != > <`

비교하면 True 아니면 False

`and` 둘 다 True · `or` 하나라도 True · `not` 뒤집기

푸는 순서는 계산, 비교, not, and, or. 헷갈리면 괄호로 묶어요